NextPlay

Documentación de funciones

134 funciones en 16 archivos · generado el 17/10/2026 23:27

Archivo	Funciones
accounts/apps.py	1
accounts/forms.py	2
accounts/models.py	25
accounts/signals.py	1
accounts/views.py	22
games/admin.py	1
games/igdb.py	15
games/management/commands/import_igdb.py	3
games/management/commands/seed_demo.py	1
games/management/commands/update_prices.py	2
games/models.py	23
games/prices.py	13
games/views.py	3
games/xbox.py	9
library/models.py	7
library/views.py	6

App «accounts»

Usuarios: registro, inicio de sesión, perfil público o privado, vinculación con Steam, Xbox y PlayStation y cuentas de otras plataformas.

accounts/apps.py

Clase AccountsConfig

```
def ready(self)
```

Registra las señales de la app al arrancar Django (crean el perfil de cada usuario nuevo).

accounts/apps.py, línea 7

accounts/forms.py

Clase SignupForm

```
def clean_email(self)
```

Pasa el correo a minúsculas y rechaza el registro si ya hay una cuenta con ese correo.

accounts/forms.py, línea 15

Clase PlatformAccountForm

```
def clean(self)
```

Normaliza el nombre o código y comprueba que tiene el formato de la plataforma elegida.

accounts/forms.py, línea 34

accounts/models.py

Clase Profile

```
def __str__(self)
```

Texto del perfil en el admin y en la consola: su nombre visible.

accounts/models.py, línea 25

```
@property
def name(self)
```

Nombre que se muestra: el nombre visible si lo tiene, si no el nombre de usuario.

accounts/models.py, línea 30

```
def get_absolute_url(self)
```

URL del perfil público del usuario (/u/<usuario>/).

accounts/models.py, línea 34

```
def visible_to(self, viewer)
```

Indica si viewer puede ver el perfil: siempre si es público, y si es privado solo su dueño.

accounts/models.py, línea 38

```
@property
def steam_accounts(self)
```

Cuentas de Steam vinculadas, de la primera a la última (aprovecha el prefetch si lo hay).

accounts/models.py, línea 43

```
@property
def steam_linked(self)
```

Indica si el usuario tiene al menos una cuenta de Steam vinculada.

accounts/models.py, línea 48

```
@property
def steam_avatar(self)
```

Avatar de la primera cuenta de Steam vinculada, o cadena vacía si no tiene ninguna.

accounts/models.py, línea 53

```
@property
def psn(self)
```

Cuenta de PlayStation vinculada, o None si no tiene.

accounts/models.py, línea 59

```
@property
def platforms(self)
```

Cuentas de otras plataformas (Xbox, Switch, Epic...) que muestra en su perfil.

accounts/models.py, línea 67

```
@property
def xbox(self)
```

Cuenta de Xbox vinculada, o None si no tiene.

accounts/models.py, línea 72

```
@property
def avatar_url(self)
```

Avatar del perfil: el de Steam, si no el de PlayStation, si no el de Xbox; vacío para usar la inicial.

accounts/models.py, línea 80

Clase SteamAccount

Cuenta de Steam vinculada a un usuario; un usuario puede tener varias.

```
def __str__(self)
```

Texto de la cuenta en el admin: su nombre en Steam, o el SteamID si no se conoce.

accounts/models.py, línea 109

```
@property
def name(self)
```

Nombre que se muestra: el de Steam, o el SteamID si aún no se ha podido leer.

accounts/models.py, línea 114

```
@property
def url(self)
```

URL del perfil en Steam; la construye con el SteamID si Steam no la ha dado.

accounts/models.py, línea 119

Clase PSNAccount

Cuenta de PlayStation Network vinculada a un usuario (una por usuario).

No se guarda ningún código ni token de Sony: el usuario pega su código NPSSO cada vez que sincroniza.

```
def __str__(self)
```

Texto de la cuenta en el admin: su PSN ID, o el ID de cuenta si no se conoce.

accounts/models.py, línea 142

```
@property
def name(self)
```

Nombre que se muestra: el PSN ID, o «PlayStation» si Sony no lo ha devuelto.

accounts/models.py, línea 147

```
@property
def url(self)
```

Perfil público en PSNProfiles (Sony no tiene página pública de perfiles), o vacío sin PSN ID.

accounts/models.py, línea 152

Clase PlatformAccount

Cuenta de otra plataforma que el usuario muestra en su perfil (sin importar juegos).

No se verifica que sea suya: es lo que escribe, como en la mayoría de webs. Una por plataforma.

```
def __str__(self)
```

Texto de la cuenta en el admin: «Plataforma: nombre».

accounts/models.py, línea 207

```
@property
def logo_template(self)
```

Ruta de la plantilla SVG con el logo de la plataforma.

accounts/models.py, línea 212

```
@property
def url(self)
```

Perfil público en la web de la plataforma cuando existe (solo GOG); si no, vacío.

accounts/models.py, línea 217

```
@classmethod
def normalize(cls, platform, handle)
```

Limpia lo que escribe el usuario: quita espacios de los extremos y pone el código de Switch en formato SW-1234-5678-9012 aunque lo escriba solo con números o con espacios.

accounts/models.py, línea 224

```
@classmethod
def is_valid_handle(cls, platform, handle)
```

Indica si el nombre o código cumple el formato de esa plataforma.

accounts/models.py, línea 235

Clase XboxAccount

Cuenta de Xbox (cuenta Microsoft) vinculada a un usuario (una por usuario).

No se guarda ningún token de Microsoft: para sincronizar el usuario vuelve a iniciar sesión.

```
def __str__(self)
```

Texto de la cuenta en el admin: su gamertag.

accounts/models.py, línea 258

```
@property
def name(self)
```

Nombre que se muestra: el gamertag, o «Xbox» si Microsoft no lo ha devuelto.

accounts/models.py, línea 263

```
@property
def url(self)
```

Perfil público del gamertag en xbox.com, o vacío si no se conoce.

accounts/models.py, línea 268

accounts/signals.py

```
@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_profile(sender, instance, created, raw=False, **kwargs)
```

Crea el perfil de un usuario justo después de crearlo.

No hace nada al guardar un usuario ya existente ni al cargar fixtures (raw=True).

accounts/signals.py, línea 9

accounts/views.py

```
def signup(request)
```

Registro de cuenta nueva.

Si el usuario ya ha iniciado sesión lo manda a su perfil. Al registrarse bien inicia su sesión y lo lleva al catálogo para que añada sus primeros juegos.

accounts/views.py, línea 40

```
def profile(request, username)
```

Perfil público de un usuario: cabecera con estadísticas, su biblioteca y sus reseñas.

La biblioteca tiene pestañas por estado (Jugando, Completado...) con su contador y se filtra con ?estado=; se ordena con ?orden= (recientes, nota, horas o nombre). Con ?vista=resenas muestra sus reseñas en vez de la biblioteca. Un perfil privado da 404 a cualquiera que no sea su dueño.

accounts/views.py, línea 57

```
def _profile_stats(owner, entries, counts)
```

Cifras de la cabecera del perfil: juegos, completados, horas jugadas (Steam + PlayStation + Xbox), nota media y reseñas.

También calcula los tres géneros más repetidos de su biblioteca.

accounts/views.py, línea 127

```
@login_required
def edit_profile(request)
```

Formulario para que el usuario con sesión cambie su nombre visible, biografía y privacidad.

accounts/views.py, línea 147

```
def linking_context()
```

Variables que necesitan los paneles de vinculación de cuentas (en Editar perfil y en el modal del perfil).

accounts/views.py, línea 161

```
def _back(request, default)
```

Redirige a la página de la que vino el formulario (campo «next») si es de esta web; si no, a default.

Así las acciones lanzadas desde el modal del perfil vuelven al perfil y no a Editar perfil.

accounts/views.py, línea 176

```
def _steam_return_to(request, state)
```

URL absoluta a la que Steam devuelve al usuario tras iniciar sesión, con el código anti-CSRF state.

accounts/views.py, línea 187

```
def _report_import(request, result)
```

Avisa al usuario de cuántos juegos se han importado de Steam, cuántos no estaban en el catálogo y qué cuentas se han saltado por ser privadas.

accounts/views.py, línea 192

```
def _import_steam(request, include_unplayed=False)
```

Lanza la importación de todas las cuentas de Steam del usuario y le avisa del resultado o del fallo.

Si falla por Steam o IGDB deja el error completo en el log del servidor para poder revisarlo.

accounts/views.py, línea 208

```
@require_POST
@login_required
def steam_link(request)
```

Manda al usuario a iniciar sesión en Steam para vincular una cuenta más (Steam nunca nos da su contraseña).

No deja pasar de SteamAccount.MAX_PER_USER cuentas.

accounts/views.py, línea 227

```
@login_required
def steam_callback(request)
```

Vuelta desde Steam: verifica el inicio de sesión, añade la cuenta a las del usuario e importa sus juegos.

Rechaza la vuelta si el código anti-CSRF no coincide, si Steam no la confirma o si esa cuenta de Steam ya está vinculada a otro usuario. Si ya era suya, solo avisa.

accounts/views.py, línea 240

```
@require_POST
@login_required
def steam_import(request)
```

Vuelve a importar los juegos de todas las cuentas de Steam vinculadas (con «unplayed» también los no jugados).

accounts/views.py, línea 282

```
@require_POST
@login_required
def steam_unlink(request, account_id)
```

Desvincula una cuenta de Steam del usuario; los juegos importados se quedan en su biblioteca.

Solo puede desvincular cuentas suyas: con el ID de una cuenta ajena responde 404.

accounts/views.py, línea 296

```
def _psn_error(request, text)
```

Avisa del error de PlayStation y vuelve a donde estaba el formulario (por defecto, Editar perfil).

accounts/views.py, línea 309

```
@require_POST
@login_required
def psn_sync(request)
```

Vincula la cuenta de PlayStation del código NPSSO pegado (o la actualiza) e importa sus juegos de PS4/PS5.

El código solo se usa durante esta petición: no se guarda. Avisa si el código no tiene el formato esperado, si Sony no lo acepta o si esa cuenta de PlayStation ya es de otro usuario.

accounts/views.py, línea 317

```
@require_POST
@login_required
def psn_unlink(request)
```

Desvincula la cuenta de PlayStation del usuario; los juegos importados se quedan en su biblioteca.

accounts/views.py, línea 352

```
@require_POST
@login_required
def platform_add(request)
```

Añade la cuenta de otra plataforma (Xbox, Switch, Epic, GOG, Battle.net) o cambia la que ya tenía.

Si el formato no es el de esa plataforma avisa del error y no guarda nada.

accounts/views.py, línea 361

```
@require_POST
@login_required
def platform_remove(request, platform)
```

Quita del perfil del usuario su cuenta de esa plataforma (no da error si no la tenía).

accounts/views.py, línea 382

```
def _xbox_redirect_uri(request)
```

URL absoluta de vuelta desde Microsoft; tiene que coincidir con la registrada en la aplicación de Azure.

accounts/views.py, línea 392

```
@require_POST
@login_required
def xbox_link(request)
```

Manda al usuario a iniciar sesión con su cuenta Microsoft para vincular Xbox o volver a sincronizarla.

Sirve para las dos cosas porque no se guarda ningún token: cada sincronización pide iniciar sesión.

accounts/views.py, línea 399

```
@login_required
def xbox_callback(request)
```

Vuelta desde Microsoft: comprueba el código anti-CSRF, vincula la cuenta de Xbox e importa sus juegos.

Si el usuario canceló el inicio de sesión, si el código no coincide o si Xbox Live falla, avisa y vuelve a Editar perfil.

accounts/views.py, línea 412

```
@require_POST
@login_required
def xbox_unlink(request)
```

Desvincula la cuenta de Xbox del usuario; los juegos importados se quedan en su biblioteca.

accounts/views.py, línea 448

App «games»

Catálogo de juegos, importación desde IGDB y precios de PC (IsThereAnyDeal) y Xbox.

games/admin.py

Clase GamePriceInline

```
def has_add_permission(self, request, obj=None)
```

Impide añadir precios a mano: solo los escribe update_prices.

games/admin.py, línea 13

games/igdb.py

Cliente de IGDB y la función que vuelca sus juegos en el catálogo local.

Clase IGDBError

Fallo al hablar con IGDB o con Twitch (credenciales, red o respuesta no válida).

```
def is_configured()
```

Indica si están puestas las credenciales de Twitch para usar IGDB.

games/igdb.py, línea 40

Clase IGDBClient

```
def __init__(self, client_id=None, client_secret=None)
```

Prepara el cliente con las credenciales dadas o las de settings; falla si no hay ninguna.

games/igdb.py, línea 46

```
def _token(self)
```

Token de acceso de Twitch: lo saca de la caché o pide uno nuevo y lo guarda hasta que caduque.

games/igdb.py, línea 53

```
def query(self, endpoint, body)
```

Lanza una consulta en Apicalypse (el lenguaje de IGDB) contra endpoint y devuelve el JSON.

Si IGDB responde 401 borra el token de la caché para que la próxima llamada pida uno nuevo.

games/igdb.py, línea 76

```
def top_games(self, limit=MAX_PAGE_SIZE, offset=0)
```

Una página de los juegos más valorados por número de votos, sin ediciones sueltas, con sus ediciones.

games/igdb.py, línea 99

```
def search(self, term, limit=20)
```

Busca juegos por texto en IGDB (hasta limit resultados, sin ediciones sueltas), con sus ediciones.

games/igdb.py, línea 108

```
def games_by_steam_ids(self, app_ids)
```

Juegos de IGDB (con sus ediciones) de una lista de IDs de Steam y un dict {ID de Steam: id de IGDB}.

Los IDs que IGDB no conoce (herramientas, bandas sonoras, servidores...) no aparecen en el dict.

games/igdb.py, línea 117

```
def games_by_names(self, names)
```

Juegos de IGDB (con sus ediciones) cuyo nombre es exactamente uno de names, sin ediciones sueltas.

Va en lotes para no hacer una petición por juego. Si dos juegos se llaman igual vienen los dos; quien llama decide con cuál quedarse (p. ej. el más popular).

games/igdb.py, línea 142

```
def _with_editions(self, games)
```

Añade a cada juego sus ediciones (GOTY, Definitive...) en la clave «editions», para tener sus IDs de tienda.

games/igdb.py, línea 161

```
def _upsert_ref(model, data, **extra)
```

Busca o crea un Genre o Platform de IGDB, aprovechando uno que no venga de IGDB con el mismo slug.

games/igdb.py, línea 183

```
def _normalize(name)
```

Nombre para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

games/igdb.py, línea 199

```
def _store_ids(data)
```

IDs de tienda de un juego de IGDB: {"steam": [...], "gog": [...]}.

IGDB puede tener varias entradas por tienda: relanzamientos con el mismo nombre que el juego (se quedan todos y sus ofertas se juntan) y otras raras como «Mac Portal» (solo se usa la primera, y únicamente si ninguna se llama como el juego).

games/igdb.py, línea 204

```
def _store_alternatives(data, own_ids)
```

IDs de tienda de las ediciones del juego y después de sus remasters, sin repetir los suyos.

Sirven para encontrar precios cuando el juego original ya no se vende.

games/igdb.py, línea 225

```
def _xbox_product_ids(data)
```

IDs de producto de Microsoft Store del juego y después de sus ediciones y remasters, sin repetidos.

games/igdb.py, línea 244

```
@transaction.atomic
def upsert_games(payload)
```

Crea o actualiza los juegos locales a partir de los de IGDB y devuelve la lista de Game.

Guarda datos, nota, portada, IDs de tienda, géneros y plataformas. Aprovecha juegos de demo o creados a mano con el mismo nombre o slug en vez de duplicarlos. Si cambian los IDs de tienda, borra los precios de PC, que se buscaron con los IDs viejos. Todo va en una transacción.

games/igdb.py, línea 257

games/management/commands/import_igdb.py

Clase Command

```
def add_arguments(self, parser)
```

Opciones del comando: --limit (cuántos populares) y --search (importar una búsqueda).

games/management/commands/import_igdb.py, línea 11

```
def handle(self, *args, **options)
```

Punto de entrada: importa una búsqueda o los más populares y dice cuántos juegos guardó.

games/management/commands/import_igdb.py, línea 16

```
def _import_top(self, client, limit)
```

Importa los limit juegos más populares en páginas de hasta 500. Devuelve cuántos importó.

games/management/commands/import_igdb.py, línea 28

games/management/commands/seed_demo.py

Clase Command

```
def handle(self, *args, **options)
```

Crea los 12 juegos de GAMES con sus géneros y plataformas; se salta los que ya existen.

games/management/commands/seed_demo.py, línea 37

games/management/commands/update_prices.py

Clase Command

```
def add_arguments(self, parser)
```

Opciones del comando: --limit (solo los N más populares) y --game (un solo juego por slug).

games/management/commands/update_prices.py, línea 11

```
def handle(self, *args, **options)
```

Actualiza los precios de PC (si hay ITAD_API_KEY) y después los de Xbox de los juegos elegidos.

games/management/commands/update_prices.py, línea 16

games/models.py

Clase Genre

```
def __str__(self)
```

Nombre del género.

games/models.py, línea 15

Clase Platform

```
def __str__(self)
```

Nombre completo de la plataforma.

games/models.py, línea 29

```
@property
def short_name(self)
```

Nombre corto para las etiquetas (p. ej. «PS5»); el completo si no tiene abreviatura.

games/models.py, línea 34

Clase Game

```
def __str__(self)
```

Nombre del juego.

games/models.py, línea 90

```
def get_absolute_url(self)
```

URL de la ficha del juego (/juegos/<slug>/).

games/models.py, línea 94

```
@property
def cover_url(self)
```

URL de la portada en el CDN de IGDB, o cadena vacía si el juego no tiene portada.

games/models.py, línea 99

```
@property
def release_year(self)
```

Año de lanzamiento, o None si no se conoce la fecha.

games/models.py, línea 106

```
@property
def price_lookups(self)
```

Grupos ([(tienda, ID en la tienda), ...], nombre de edición) con los que buscar precios, por preferencia.

Primero los IDs del propio juego y después los de sus ediciones y remasters. Los IDs de un mismo grupo son el mismo producto, así que sus ofertas se juntan. Si hay un ID de Steam manual, solo se usa ese.

games/models.py, línea 111

```
@property
def xbox_lookup_ids(self)
```

IDs de producto de Xbox con los que buscar el precio: el manual si lo hay, si no los de IGDB.

games/models.py, línea 129

```
@property
def on_current_xbox(self)
```

Indica si el juego está en Xbox One o Xbox Series X|S.

games/models.py, línea 134

```
@property
def is_free(self)
```

Indica si alguna tienda lo da gratis (precio y precio normal a 0), es decir, si es free-to-play.

games/models.py, línea 139

```
def price_groups(self)
```

Reparte los precios para la ficha del juego en (destacados, listados, plegados, mejor).

destacados: las tiendas más conocidas (Steam, Epic, Microsoft, GOG, Xbox) en un orden fijo. listados: las demás más baratas, solo si el juego no está en ninguna destacada. plegados: el resto por precio, dentro de «Ver N tiendas más». mejor: la oferta más barata cuando ha quedado plegada, para poder señalarla igualmente.

games/models.py, línea 143

```
def store_searches(self)
```

(tienda, plantilla del logo, URL de búsqueda) de las consolas en las que está el juego pero sin precio.

Son PlayStation Store, Nintendo eShop y, si no se encontró precio de Xbox, Xbox Store.

games/models.py, línea 160

```
def community_rating(self)
```

Media de las valoraciones de los usuarios de NextPlay: {"average": 1-5 o None, "count": n}.

games/models.py, línea 173

```
@property
def critic_score(self)
```

Nota de la crítica redondeada (0-100), o None si no tiene.

games/models.py, línea 180

```
@property
def critic_tier(self)
```

Franja de la nota para colorearla: «high» (75 o más), «mid» (50-74), «low» o vacío sin nota.

games/models.py, línea 185

```
@property
def historical_low_symbol(self)
```

Símbolo de la moneda del mínimo histórico (€, $, £...).

games/models.py, línea 193

```
def currency_symbol(code)
```

Símbolo de una moneda a partir de su código ISO («EUR» → «€»); el propio código si no lo conoce.

games/models.py, línea 227

Clase GamePrice

Precio actual de un juego en una tienda.

```
def __str__(self)
```

Texto del precio en el admin: «juego · tienda: precio moneda».

games/models.py, línea 255

```
@property
def symbol(self)
```

Símbolo de la moneda del precio.

games/models.py, línea 260

```
@property
def is_free(self)
```

Indica si el juego es gratis en esta tienda (no solo rebajado a 0 por una oferta).

games/models.py, línea 265

```
@property
def is_featured(self)
```

Indica si la tienda es una de las destacadas (Steam, Epic, Microsoft, GOG, Xbox).

games/models.py, línea 270

```
@property
def logo_template(self)
```

Ruta de la plantilla SVG con el logo de la tienda; solo para tiendas destacadas.

games/models.py, línea 275

games/prices.py

Cliente de IsThereAnyDeal y la sincronización que guarda los precios actuales de PC de cada juego.

Clase ITADError

Fallo al hablar con IsThereAnyDeal (clave, red o respuesta no válida).

```
def is_configured()
```

Indica si está puesta la clave de IsThereAnyDeal.

games/prices.py, línea 25

Clase ITADClient

```
def __init__(self, api_key=None, country=None)
```

Prepara el cliente con la clave y el país dados o los de settings; falla si no hay clave.

games/prices.py, línea 31

```
def _post(self, path, body, params=None)
```

POST a la API con la clave y devuelve el JSON.

Si la API responde 429 (demasiadas peticiones) espera lo que diga Retry-After (máximo 5 minutos) y reintenta hasta 3 veces.

games/prices.py, línea 38

```
def lookup(self, store, store_ids)
```

Traduce IDs de una tienda (de Steam o de GOG) a los UUID de juego de ITAD (None si no lo conoce).

games/prices.py, línea 62

```
def prices(self, itad_ids)
```

Ofertas actuales y mínimo histórico de cada juego de ITAD en el país configurado.

games/prices.py, línea 68

```
def _amount(value)
```

Importe de un precio de ITAD como Decimal con dos decimales.

games/prices.py, línea 73

```
def _batches(items)
```

Parte la lista en trozos de BATCH_SIZE, lo máximo que acepta la API por petición.

games/prices.py, línea 78

```
def _cheapest_per_shop(deals)
```

Se queda con la oferta más barata de cada tienda.

games/prices.py, línea 84

```
@transaction.atomic
def _store_prices(game, data, edition='')
```

Sustituye los precios de PC del juego por las ofertas de data y guarda su mínimo histórico.

Con data vacío deja el juego sin precios de PC. edition es el nombre de la edición de la que salen los precios (vacío si son del propio juego).

games/prices.py, línea 96

```
def _lookup(client, keys)
```

Traduce un conjunto de (tienda, ID) a UUID de ITAD, tienda por tienda y por lotes.

games/prices.py, línea 124

```
def _fetch_prices(client, itad_ids)
```

Precios de todos los UUID de ITAD pedidos por lotes: {uuid: datos de precios}.

games/prices.py, línea 134

```
def _merge(items)
```

Junta los precios de varios juegos de ITAD que son el mismo producto (p. ej. un relanzamiento).

Suma todas sus ofertas y se queda con el mínimo histórico más bajo. Devuelve None si no hay datos.

games/prices.py, línea 142

```
def update_prices(games, client=None)
```

Actualiza los precios de PC de los juegos y devuelve cuántos acabaron con precios.

Va por rondas: en la primera busca con los IDs del propio juego; los juegos que se quedan sin ofertas pasan a la siguiente ronda con su primera edición o remaster, y así sucesivamente.

games/prices.py, línea 157

games/views.py

```
def catalog(request)
```

Catálogo de juegos, que también es la portada.

Filtra por nombre (?q=), género (?genero=) y plataforma (?plataforma=) y pagina de 24 en 24. Si hay sesión, marca en cada tarjeta el estado del juego en la biblioteca del usuario. Ofrece importar de IGDB cuando hay búsqueda, sesión y credenciales de IGDB.

games/views.py, línea 18

```
def detail(request, slug)
```

Ficha de un juego: datos, nota de la crítica, valoración de la comunidad, widget de biblioteca, precios y reseñas.

Los precios llegan ya repartidos por Game.price_groups(); store_searches() da los enlaces de búsqueda a las tiendas de consola sin precio. Las reseñas de perfiles privados solo las ve su autor.

games/views.py, línea 63

```
@require_POST
@login_required
def import_search(request)
```

Importa de IGDB los resultados de una búsqueda que no estaban en el catálogo local.

Después vuelve al catálogo con la misma búsqueda y avisa de cuántos juegos se importaron, o de que IGDB no respondió.

games/views.py, línea 96

games/xbox.py

Precios de consola Xbox sacados del catálogo público de la tienda de Microsoft (sin documentar, sin clave).

Clase XboxError

Fallo al consultar la tienda de Xbox (red o respuesta no válida).

```
def _market()
```

País de la tienda, el mismo que ITAD_COUNTRY (p. ej. «ES»).

games/xbox.py, línea 24

```
def _locale()
```

Idioma y país de la tienda, p. ej. «es-ES».

games/xbox.py, línea 29

```
def fetch_products(product_ids)
```

Devuelve {ID de producto: (precio, precio normal, moneda)} de los productos que se pueden comprar en consola Xbox.

Pide los productos de 20 en 20; los que no se venden en consola no aparecen en el resultado.

games/xbox.py, línea 34

```
def _console_offer(product)
```

Oferta de compra más barata de un producto que funciona en Xbox, o None si no hay.

Ignora las de 0,00, que son Game Pass, pruebas o copias incluidas en packs.

games/xbox.py, línea 59

```
def _normalize(title)
```

Título para comparar: sin ™ ® ©, en minúsculas y con los espacios simplificados.

games/xbox.py, línea 85

```
def search_games(name)
```

IDs de producto de los juegos de Xbox que se llaman exactamente como name (títulos en inglés, como en IGDB).

games/xbox.py, línea 90

```
def store_url(product_id)
```

Enlace a la página del producto en xbox.com.

games/xbox.py, línea 117

```
@transaction.atomic
def _store(game, product_id, offer)
```

Sustituye el precio de Xbox del juego por offer y calcula el % de descuento; sin offer lo deja sin precio.

games/xbox.py, línea 123

```
def update_xbox_prices(games, sleep=time.sleep)
```

Guarda el precio de Xbox de cada juego y devuelve cuántos acabaron con precio.

Usa el primer ID de producto (el suyo y después sus ediciones) que se venda en consola. Como IGDB a menudo solo tiene productos que ya no se venden (p. ej. la versión original de un relanzamiento), los juegos de Xbox One/Series sin precio se buscan también por su título exacto, salvo que otro juego del catálogo se llame igual o que la búsqueda dé más de un producto.

games/xbox.py, línea 141

App «library»

Biblioteca de cada usuario: qué juegos tiene, en qué estado, su valoración y su reseña.

library/models.py

Clase LibraryEntryQuerySet

```
def reviews_visible_to(self, viewer)
```

Entradas con reseña que viewer puede leer, de la más reciente a la más antigua.

Las de perfiles privados solo las ve su autor, igual que su biblioteca.

library/models.py, línea 16

Clase LibraryEntry

```
def __str__(self)
```

Texto de la entrada en el admin: «usuario · juego · estado».

library/models.py, línea 81

```
@classmethod
def imported_status(cls, minutes, last_played)
```

Estado de un juego que llega de una importación (Steam, PlayStation o Xbox) según cuánto y cuándo se jugó.

Sin minutos ni partida: «Pendiente». Jugado en los últimos IMPORT_RECENT_DAYS días: «Jugando». Si no: «Jugado».

library/models.py, línea 86

```
def update_import(self, platform, minutes, last_played)
```

Guarda los minutos y la última partida de una plataforma ("steam", "psn" o "xbox") en una entrada existente.

Conserva el estado que puso el usuario, salvo «Pendiente» cuando ya hay horas jugadas: entonces lo cambia a «Jugando» o «Jugado». Devuelve True si ha cambiado algo.

library/models.py, línea 98

```
@property
def playtime_minutes(self)
```

Minutos jugados sumando Steam, PlayStation y Xbox, o None si no viene de ninguna.

library/models.py, línea 119

```
@property
def playtime_hours(self)
```

Horas jugadas (Steam + PlayStation + Xbox) redondeadas a una décima, o None si no hay datos.

library/models.py, línea 127

```
@property
def steam_hours(self)
```

Horas jugadas en Steam redondeadas a una décima, o None si no viene de Steam.

library/models.py, línea 133

library/views.py

```
def review_context(game, entry, viewer)
```

Variables de la sección de reseñas de un juego: las reseñas que viewer puede leer, cuántas son y el formulario de su propia reseña (None si el juego no está en su biblioteca).

library/views.py, línea 17

```
def _respond(request, game, entry)
```

Respuesta común de las acciones de biblioteca.

Si la petición viene de HTMX devuelve el widget de biblioteca ya actualizado junto con el recuadro de valoración de la comunidad y la sección de reseñas (que HTMX cambia en su sitio); si no, redirige a la ficha del juego.

library/views.py, línea 28

```
@require_POST
@login_required
def set_status(request, game_id)
```

Añade el juego a la biblioteca del usuario con el estado enviado, o cambia el que tenía.

Responde 400 si el estado no es uno de los permitidos.

library/views.py, línea 55

```
@require_POST
@login_required
def remove(request, game_id)
```

Quita el juego de la biblioteca del usuario (no da error si no estaba).

library/views.py, línea 72

```
@require_POST
@login_required
def rate(request, game_id)
```

Guarda la valoración del usuario (1 a 5 estrellas) para un juego de su biblioteca.

Con rating=0 borra la valoración. Responde 404 si el juego no está en su biblioteca y 400 si la valoración no es un número del 0 al 5.

library/views.py, línea 81

```
@require_POST
@login_required
def review(request, game_id)
```

Guarda, cambia o borra (si llega vacía) la reseña del usuario sobre un juego de su biblioteca.

Responde 404 si el juego no está en su biblioteca. Si la reseña no es válida (demasiado larga) avisa del error. Siempre vuelve a la sección de reseñas de la ficha del juego.

library/views.py, línea 102